



Welcome

—
recording — let's go



Week 5

—
Verification

what changed this week?

SemiAnalysis on OpenAI's **Jalapeño** inference chip, and tokens per *joule* is the key metric

ICML keynote by Arvind Narayanan **What** will be left for us to work on?

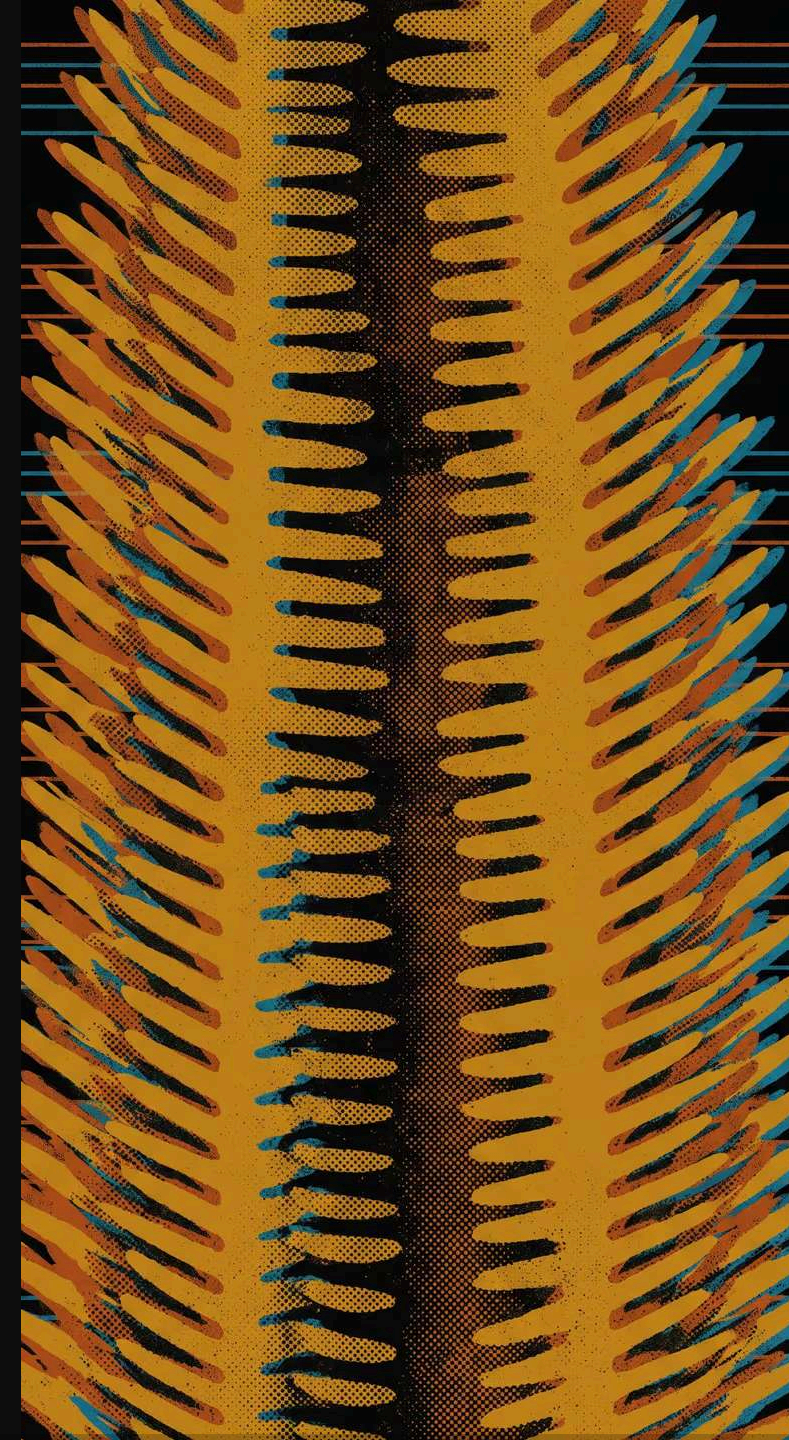


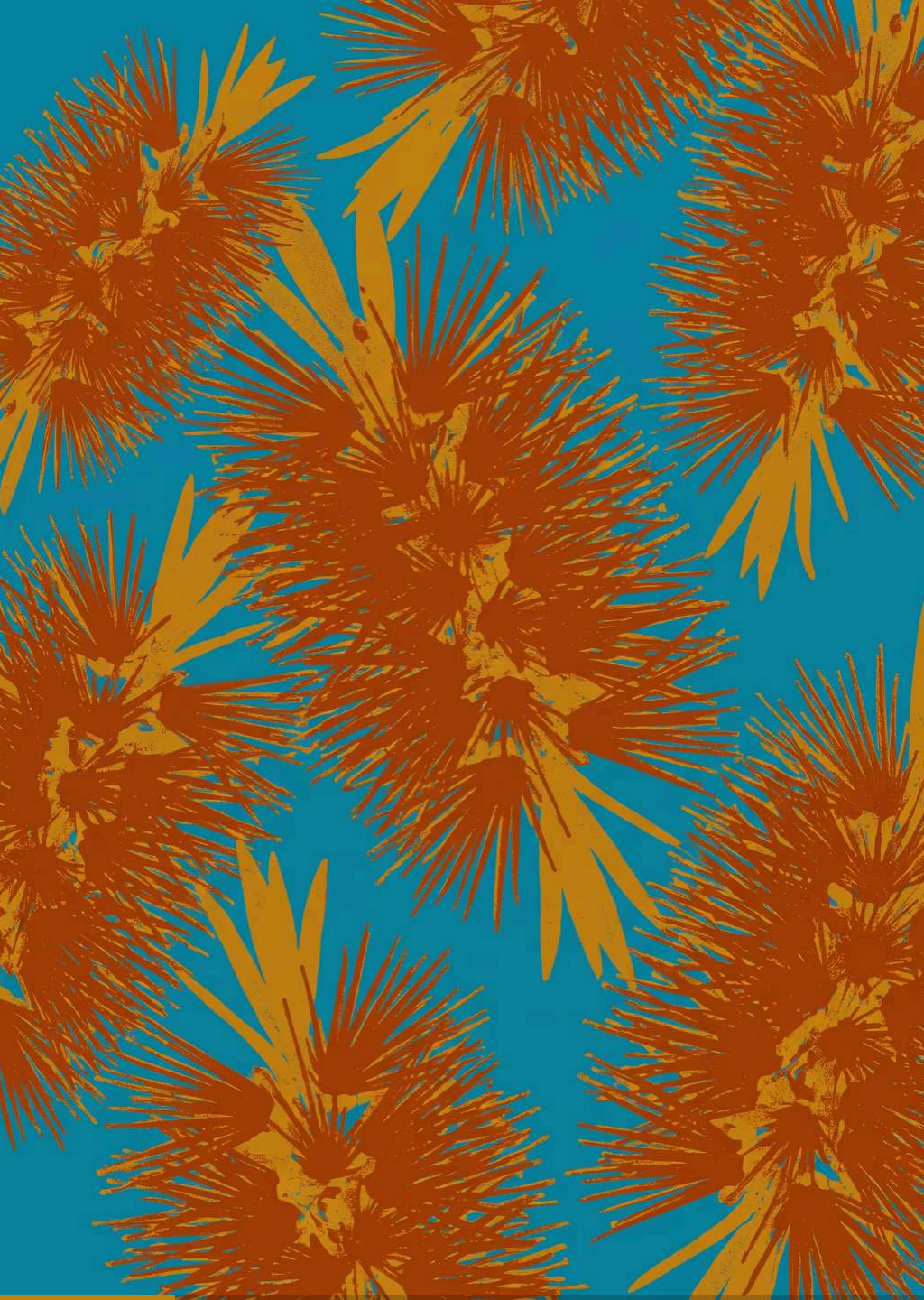
today's beats

never assume it works until it has run

cognitive debt, agentic coding and you

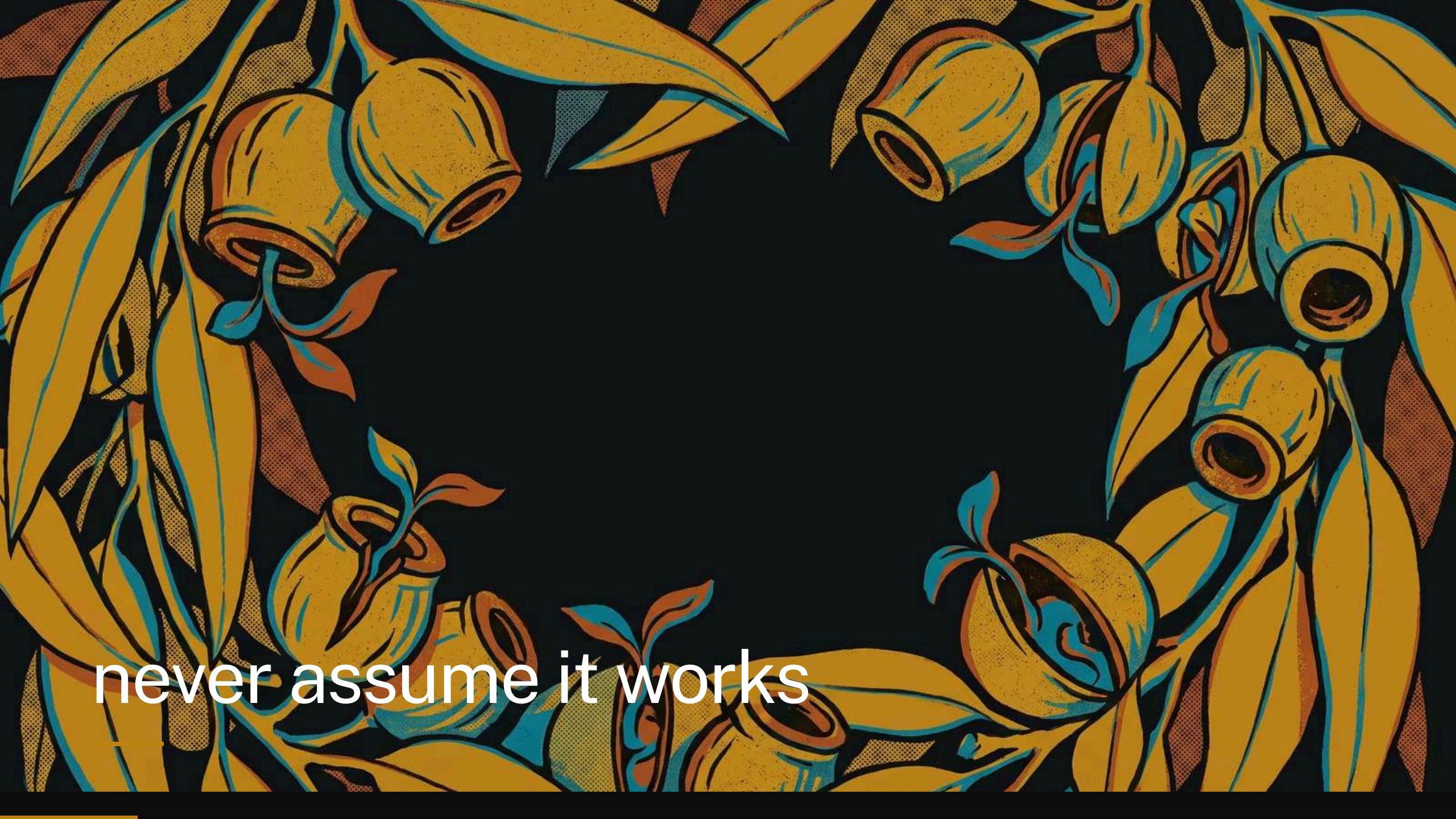
assignment 2 released





demo: cold music-making

then we leave it running and talk about what
verifying actually means



never assume it works

instruct, then verify

Simon Willison:

the skill is confidently **instructing** agents, then
confidently **verifying** the results

weeks 2 and 3 were the instructing half (brief,
spec, plan, harness), this is the other half



a passing suite is not a working feature

never assume that code generated by an LLM
works until that code has been executed

it can pass every test and still crash on startup,
render nothing, or work only on the 'golden
path'



three patterns

first run the tests: get a green baseline before you ask for anything

agentic manual (unscripted) testing: drive the running artefact, report what you saw

red/green TDD: anything the drive-through found becomes a permanent test

ref: [Agentic Engineering Patterns](#)





words; what do they even mean?

first run the tests

```
$ pnpm test
✓ spec/invariants.test.ts (4 tests) 812ms
✓ spec/tides.test.ts (3 tests) 141ms

Test Files  2 passed (2)
Tests      7 passed (7)
```

thirty seconds at the top assures you that a red result later belongs to *your change*

agentic manual testing

```
# --no-sandbox necessary on linux with apparmor  
$ agent-browser --args "--no-sandbox" open http://localhost:4321/
```

```
console  TypeError: Cannot read properties of null  
         (reading 'addEventListener')  
         at initTides (tides.js:14:22)
```

same commit as the green suite above

the tests exercised the functions; nobody had opened the page

make it show its work

I tested the form and it submits correctly

```
$ curl -s -X POST localhost:4321/api/tide -d 'port=hobart'
{"error": "unknown port"}
```

the agent will happily write up what it *hoped* had happened

ask for the artefact: the screenshot, the console, the response body, the exit code

manual testing finds it once

nothing stops the bug coming back on the next change

so close it properly: failing test first, watch it fail for the right reason, then fix



the suite verified the artefact

nothing verified **you**



cognitive debt

when we lose track of how code written by our agents works we take on **cognitive debt**

Margaret-Anne Storey's term

not always necessary, so pay it down where you think it'll be worth it (the thing you'd have to debug an hour before a crit)



Ben uses... “simplify”
requests

maybe I should make it a slash command...



find out before your pod does

say it out loud: describe the change you just accepted without looking

make it examine you: ask the agent to quiz you on the code it just wrote

your pod runs the first one on you this week anyway



the first remedy is one prompt

```
> walk me through src/tides.js in the order  
  I'd need to read it to change the  
  interpolation – entry point first, and  
  what each part is there for
```

that's a **linear walkthrough**: a structured tour, in the order a reader needs rather than the order the files sit in

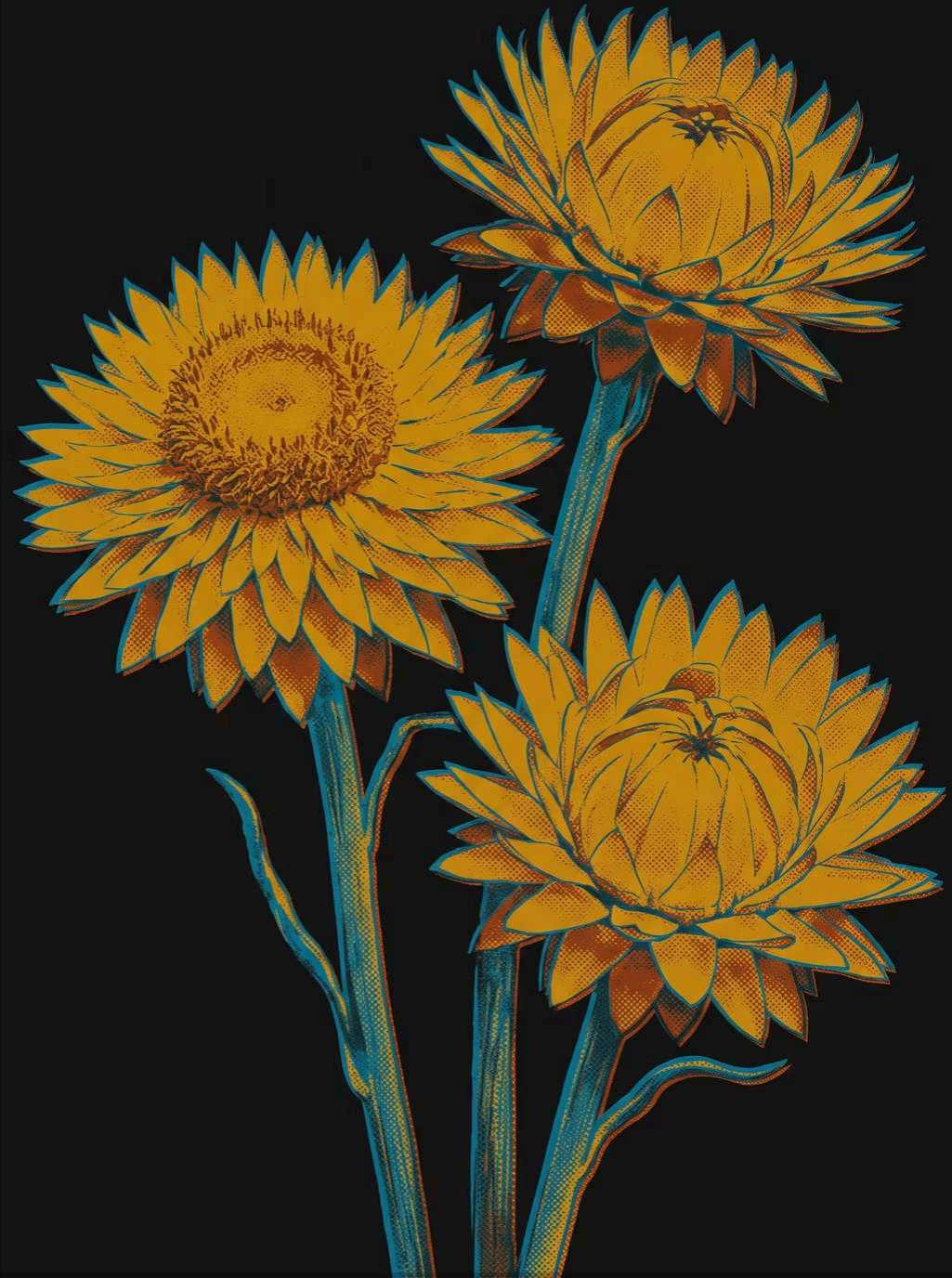
point it at your agent's code, or at the library you're about to depend on



you have already built the second one

interactive explanations: an explorable, for when the tour still leaves you without intuition

Willison's example is a word-cloud placement algorithm whose description told him nothing until he had an animation with a speed slider



Ben uses... pristine output

pristine output means zero failures, errors, warnings, and backtraces

another standing line in my

```
~/ .claude/CLAUDE.md
```

because if there's junk in the log I miss the stuff I should care about



yes, and...

how does it get pushed further? weird ideas welcome



drive it the way a marker will

both viewports: week 2's device toolbar, still the phone that breaks

the keyboard: tab through it without touching the mouse

the console open: a page can look completely fine and be throwing on every interaction



looks fine, throws anyway

```
GET /assets/tide-data.json 404 (Not Found)  
Uncaught (in promise) SyntaxError: Unexpected  
token '<', "<!DOCTYPE "... is not valid JSON
```

the page rendered, the layout is perfect, the screenshot looks great

the server answered the missing file with its 404 page, the fetch parsed HTML, and every number on screen is a placeholder

the screenshot passed and the console didn't, which is why both are on the list



it can actually see it

Claude is multimodal, and so is every other frontier model: a screenshot gets *read*, not filed as an attachment

it can give useful feedback, even on design stuff, but it's less of a 'right/wrong' thing



the stranger test

C5's no-tutorial rule is the part you cannot put under test and cannot fake

that's not a shortfall of your harness — it's the thing the crit format exists to do



what you can't test
is what your **pod** is for



yes, and...

how does it get pushed further? weird ideas welcome



the course skills got cheaper

every skill in the `comp4020` plugin now names
a model and an effort level in its own
frontmatter

```
name: radar  
model: sonnet  
effort: low
```



`/model` and `/effort`,
decided once

week 4 called these the two cost levers

Opus runs about **7x** Sonnet per request, and
nothing `/radar`, `/balance` or `/handbook`
does needs it

(your status line now displays an Opus model
out in magenta while you're on course credits)



assignment 2 is out

design the university course you wish you
could take, and build the website that runs it

worth 20%, due noon Monday 21 September —
the first day back after the break

no (separate) reflection *doc* this time

assignment 1 was one page with one
interaction; this is twenty-odd pages that have
to agree with each other



Slop University

research-led, metrics-forward, and
continuously improving

1st in the world for research throughput (*The
World University Index*, 2026)

<https://slop.university>



what's fixed, what's yours

fixed: the Slop identity, the content collections, and the generated API that the cohort-wide programs-and-courses page ingests

yours: the pages, the decks, the components, the navigation, the visual treatment and every word

your course code's last three digits arrive with the repo and belong to nobody else in the cohort; you pick the level digit



make it niche

narrow enough that no real university would run it, with enough depth to be taught for a full semester

a course that would pass a curriculum committee unnoticed is too broad, and COMP4020 with the nouns find-and-replaced is too low-effort

sincere, deadpan and satirical all work — it still has to be a real course website

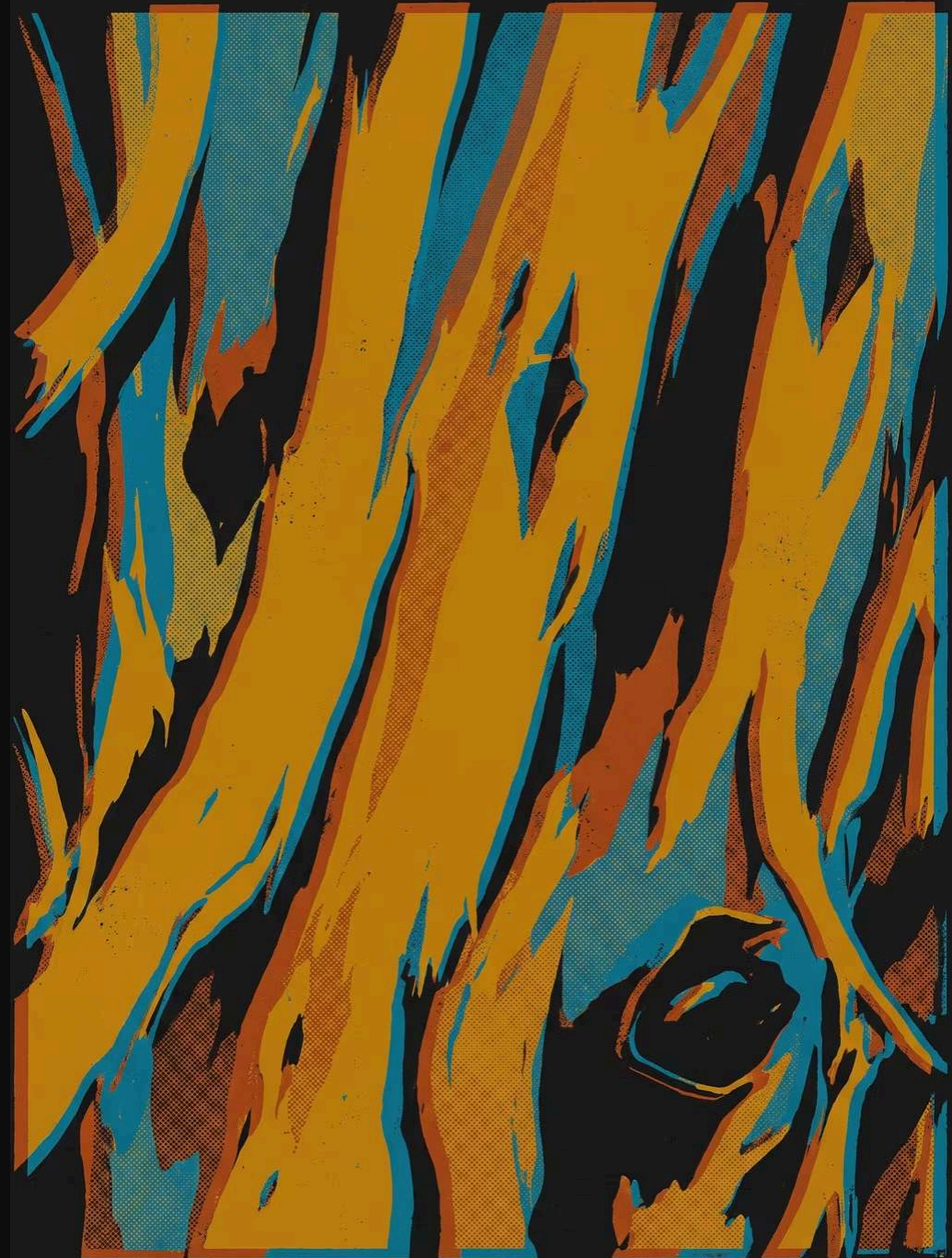


one idea, held for a semester

Calling Bullshit — a point of view stated in the title and held all semester

How to Make (Almost) Anything — a syllabus that is a list of machines, and a course that is the sum of them

CS 007 — personal finance for engineers: narrow audience, plain register, no filler

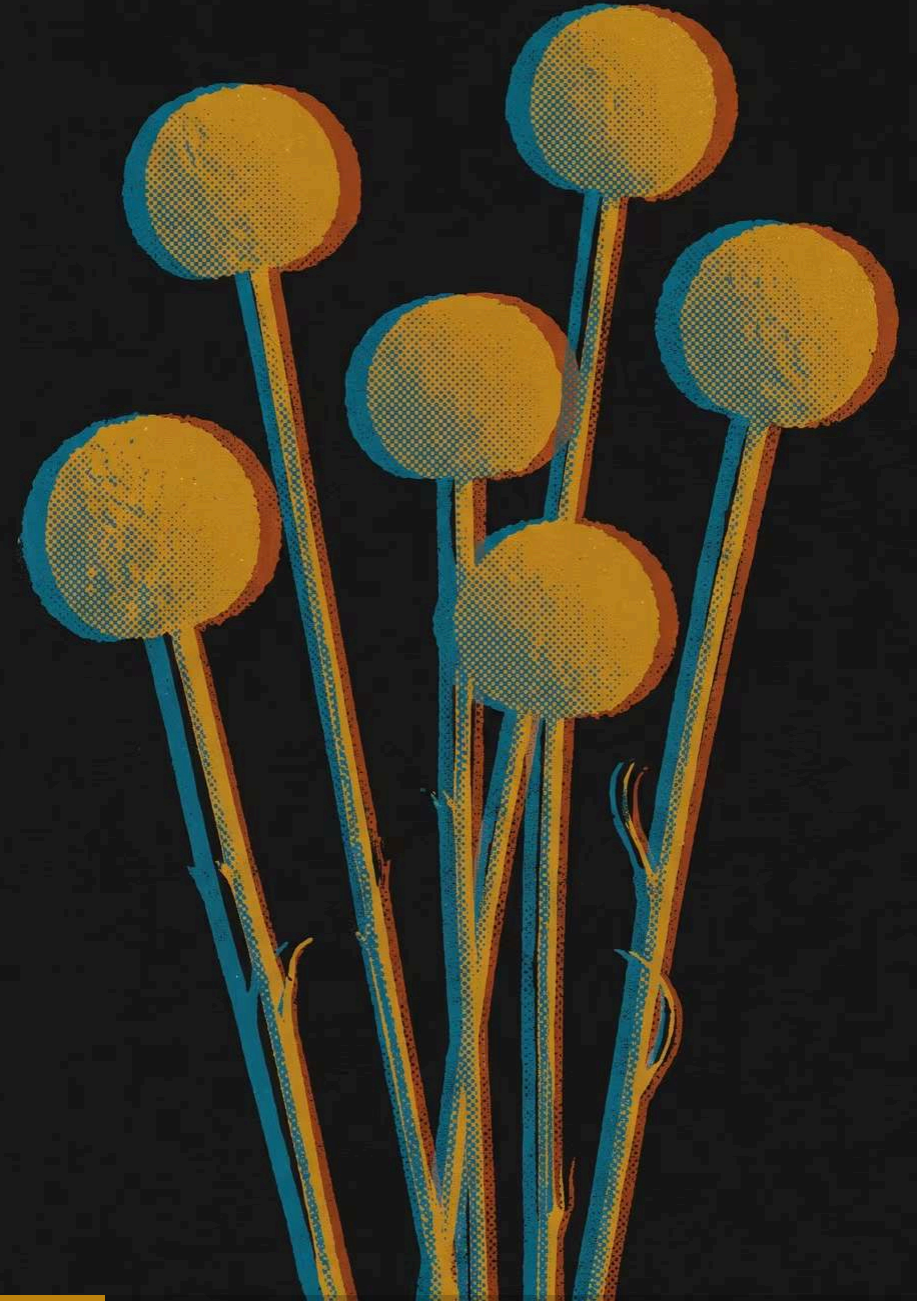


your course makes promises

yours promises particular things: assessment that adds up to 100%, every teaching week inside the semester, no session citing a lecture you never wrote

which of those deserve a check is this morning's lesson at site scale — `spec/` is submitted work, read the way `PROCESS.md` is

an agent will write you twelve weeks in a minute, and no check reaches whether they add up to a course anyone would take



the repo lands next week

provisioned early next week, then

`/comp4020:start` clones it like every other deliverable

a working Slop course site: the content model, a real deck, placeholder content and one data-integrity check — and placeholder artwork that `pnpm check:evidence` will not pass

the **brief** is live now — read it before the repo shows up



ass1 marks out tomorrow

on canvas

see **feedback and re-marks** process

<https://isgithubcooked.com/>



too vague?



the world is vague

marking this course is the same problem you have

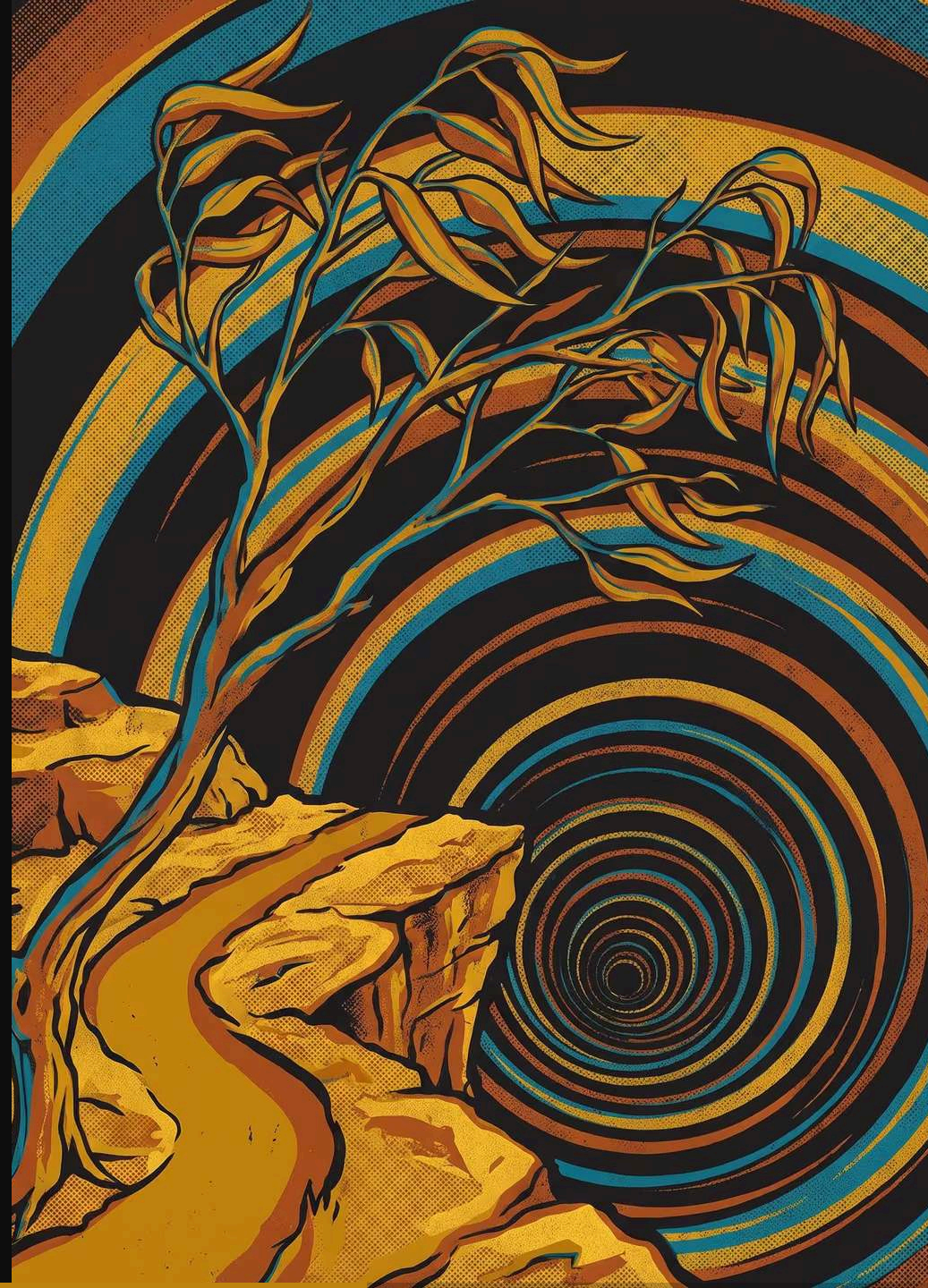
every rubric line and every spec line has to survive the fact that we now have machines which eat specs for breakfast

dealing with vagueness is our raison d'être



the existential bit

don't make it a load-bearing part of your personality



These best practices are still being figured out across our industry. I'm still figuring them out myself.

Simon Willison, Agentic Engineering Patterns



three diagnoses, one symptom

Böckeler: a sensor can't tell which convention is critical and which is just habit

Ronacher: each change is reasonable in isolation, the code compiles, the tests pass — and the coherence goes anyway

Osmani: agents can implement; they can't judge whether it's the right design

the hard case

Gas Town: Yegge's agent orchestrator — 75k lines in 17 days, dozens of agents at once, thousands of dollars a month in API spend

he describes it as complicated “not because I wanted it to be, but because I had to keep adding components until it was a self-sustaining machine”

Klabnik calls it “*aggressively* not rigorous” — and is glad someone is running the experiment

(week 9 is where claims like these go under the microscope)

a spec a machine can satisfy

when a measure becomes a target, it ceases to be a good measure

Goodhart's law, in Marilyn Strathern's phrasing
— and an unambiguous, machine-checkable
criterion is the easiest target there is

your spec is the measure and your agent is the
machine



where the weight sits

process 45%: can a reader see how you got from the brief to the thing?

artefact 20%: does the deployed thing work, and is it good?

response 35%: is the response to the brief any good?



humans making things for other **humans**



you will be asked this

tell me what value humans still bring to the software development process — examples if you like

“I use Claude Code” is not an answer to it



yes, and...

how does it get pushed further? weird ideas welcome

what will a stranger try first?



C5: a game

build a tiny browser game — one mechanic is usually enough — obvious in ten seconds, still interesting at five minutes, no tutorial in sight

last week: no way to be played wrong; this week: rules, stakes and an ending

one rule under a focused test, and one change that came from *playing* the finished game



deadline radar

Monday/Wednesday: C5 live at its URL two hours before your session, with the reflection in the repo

Monday 31 August: census date

noon, Monday 21 September: assignment 2 — the Monday you come back from the break, so it's closer than it looks



tl;dr

never assume it works until it has run — and
get the agent to show you, not tell you

cognitive debt is the failure a green suite can't
see — explain it with the file shut, then walk it
back one prompt at a time

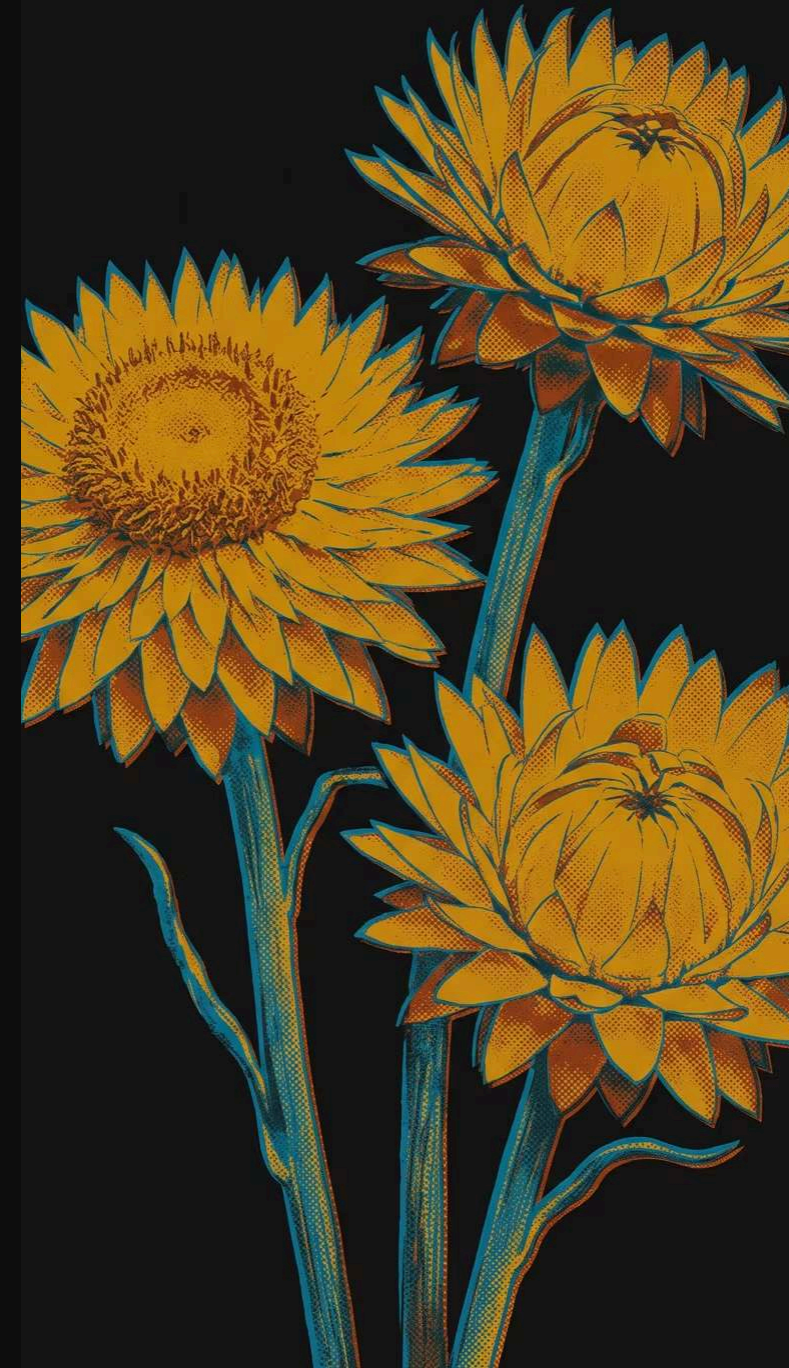
both viewports, the keyboard, the console —
then hand it to a stranger



ask your agent

ask until you can explain these back:

- **agentic manual testing:** how to drive the running artefact like a user, and what evidence to bring back
- **red/green TDD:** turning one drive-through finding into a permanent test
- **cognitive debt:** which part of your own prototype you could no longer explain to your pod
- **linear walkthroughs:** get one for a file you inherited and didn't write





See you in the studio